

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code:

```
geya@LenovoL480: ~/OS/Pra x + v
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        return 0;
    }
    else {
        printf("Parent: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent is sleeping. Check the process table now.\n");

        sleep(15);

        wait(NULL);

        printf("Parent collected the child.\n");
    }

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 4$ ./task2
Parent: PID = 785
Child PID = 786
Parent is sleeping. Check the process table now.
Child: PID = 786
Child is exiting...
Parent collected the child.
geya@LenovoL480:~/OS/Practical/Week 4$ |
```